

1. Create a SQLite database named university_assignment.db with students, courses, and enrollments tables.

```
import sqlite3
con = sqlite3.connect("university_assignment.db")
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON;")
cur.execute("""
CREATE TABLE IF NOT EXISTS students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    major TEXT
);
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS courses (
    course_code TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    credits INTEGER
);
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_code TEXT NOT NULL,
    grade TEXT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_code) REFERENCES courses(course_code)
);
""")
con.commit()
print("Database and tables created successfully.")
con.close()
```

Output:

```
In [8]: runfile('D:/TyBsc39/TY/Assign2 Q1.py', wdir='D:/TyBsc39/TY')
Database and tables created successfully.
```

2. Write a Python program to insert sample data into the students, courses, and enrollments tables using executemany() and commit the changes.

```
import sqlite3
con = sqlite3.connect("university_assignment.db")
cur = con.cursor()
cur.execute("PRAGMA foreign_keys = ON;")

students = [
    ("Rahul Sharma", "rahul@gmail.com", "Computer Science"),
    ("Priya Patel", "priya@gmail.com", "Information Technology"),
    ("Amit Verma", "amit@gmail.com", "Computer Science"),
    ("Neha Shah", "neha@gmail.com", "Mechanical"),
    ("Rohan Mehta", "rohan@gmail.com", "Information Technology")
]

cur.executemany(
    "INSERT INTO students (name, email, major) VALUES (?, ?, ?)", students)

print("Students inserted successfully.")
print("Rows inserted:", cur.rowcount)
print()
courses = [
    ("CS101", "Python Programming", 4),
    ("MA201", "Engineering Mathematics", 3),
    ("DB301", "Database Management System", 4)
]

cur.executemany(
    "INSERT INTO courses (course_code, title, credits) VALUES (?, ?, ?)", courses)

print("Courses inserted successfully.")
print("Rows inserted:", cur.rowcount)
print()
enrollments = [
    (1, "CS101", "A"),
    (1, "MA201", "B+"),
    (2, "CS101", "Pending"),
    (2, "DB301", "A"),
    (3, "CS101", "B"),
    (3, "DB301", "Pending"),
    (4, "MA201", "C"),
    (5, "DB301", "A+")
]

cur.executemany(
    "INSERT INTO enrollments (student_id, course_code, grade) VALUES (?, ?, ?)", enrollments)
print("Enrollments inserted successfully.")
print("Rows inserted:", cur.rowcount)
print()
con.commit()
```

```
print("All records inserted and committed successfully.")  
print("Last Enrollment ID:", cur.lastrowid)  
con.close()
```

Output:

```
In [9]: runfile('D:/TyBsc39/TY/Assign2 Q2.py', wdir='D:/TyBsc39/TY')  
Students inserted successfully.  
Rows inserted: 5  
  
Courses inserted successfully.  
Rows inserted: 3  
  
Enrollments inserted successfully.  
Rows inserted: 8  
  
All records inserted and committed successfully.  
Last Enrollment ID: 0
```

3. Write a Python program to retrieve and display student, course, and enrollment data from the SQLite database using SQL queries and JOINS.

```
import sqlite3

con = sqlite3.connect("university_assignment.db")
cur = con.cursor()

print("===== ALL STUDENTS =====")
cur.execute("SELECT * FROM students")
students = cur.fetchall()

for student in students:
    print(student)

print("\n===== ALL COURSES =====")
cur.execute("SELECT * FROM courses")
courses = cur.fetchall()

for course in courses:
    print(course)

print("\n===== STUDENT BY EMAIL =====")
email = "rahul@gmail.com"

cur.execute("SELECT * FROM students WHERE email = ?", (email,))
student = cur.fetchone()

if student:
    print(student)
else:
    print("Student not found.")

print("\n===== STUDENTS OF COMPUTER SCIENCE =====")

major = "Computer Science"

cur.execute("SELECT * FROM students WHERE major = ?", (major,))
records = cur.fetchall()

for record in records:
    print(record)

print("\n===== COURSES OF STUDENT (Student ID = 1) =====")

cur.execute("""
SELECT students.name,
       courses.course_code,
       courses.title,
       enrollments.grade
FROM enrollments
JOIN students
ON enrollments.student_id = students.student_id
JOIN courses
ON enrollments.course_code = courses.course_code
""")
```

```
WHERE students.student_id = ?
""", (1,))

records = cur.fetchall()

for record in records:
    print(record)

print("\n===== STUDENTS ENROLLED IN CS101 =====")

cur.execute("""
SELECT students.name,
       students.email
FROM enrollments
JOIN students
ON enrollments.student_id = students.student_id
WHERE enrollments.course_code = ?
""", ("CS101",))

records = cur.fetchall()

for record in records:
    print(record)

print("\n===== STUDENTS WITH PENDING GRADE =====")

cur.execute("""
SELECT students.name,
       students.email,
       enrollments.course_code
FROM enrollments
JOIN students
ON enrollments.student_id = students.student_id
WHERE enrollments.grade = 'Pending'
""")

records = cur.fetchall()

for record in records:
    print(record)

print("\n===== STUDENT COUNT PER MAJOR =====")

cur.execute("""
SELECT major,
       COUNT(*) AS Total_Students
FROM students
GROUP BY major
""")

records = cur.fetchall()

for record in records:
    print(record)
```

con.close()

Output:

```
In [10]: runfile('D:/TyBsc39/TY/Assign2 Q3.py', wdir='D:/TyBsc39/TY')
===== ALL STUDENTS =====
(1, 'Rahul Sharma', 'rahul@gmail.com', 'Computer Science')
(2, 'Priya Patel', 'priya@gmail.com', 'Information Technology')
(3, 'Amit Verma', 'amit@gmail.com', 'Computer Science')
(4, 'Neha Shah', 'neha@gmail.com', 'Mechanical')
(5, 'Rohan Mehta', 'rohan@gmail.com', 'Information Technology')

===== ALL COURSES =====
('CS101', 'Python Programming', 4)
('MA201', 'Engineering Mathematics', 3)
('DB301', 'Database Management System', 4)

===== STUDENT BY EMAIL =====
(1, 'Rahul Sharma', 'rahul@gmail.com', 'Computer Science')

===== STUDENTS OF COMPUTER SCIENCE =====
(1, 'Rahul Sharma', 'rahul@gmail.com', 'Computer Science')
(3, 'Amit Verma', 'amit@gmail.com', 'Computer Science')

===== COURSES OF STUDENT (Student ID = 1) =====
('Rahul Sharma', 'CS101', 'Python Programming', 'A')
('Rahul Sharma', 'MA201', 'Engineering Mathematics', 'B+')

===== STUDENTS ENROLLED IN CS101 =====
('Rahul Sharma', 'rahul@gmail.com')
('Priya Patel', 'priya@gmail.com')
('Amit Verma', 'amit@gmail.com')

===== STUDENTS WITH PENDING GRADE =====
('Priya Patel', 'priya@gmail.com', 'CS101')
('Amit Verma', 'amit@gmail.com', 'DB301')

===== STUDENT COUNT PER MAJOR =====
('Computer Science', 2)
('Information Technology', 2)
('Mechanical', 1)
```

4. Write a Python program to update student and enrollment records in the SQLite database and commit the changes.

```
import sqlite3

con = sqlite3.connect("university_assignment.db")
cur = con.cursor()

cur.execute("PRAGMA foreign_keys = ON;")

print("===== UPDATE STUDENT MAJOR =====")

student_name = "Rahul Sharma"
new_major = "Data Science"

cur.execute("""
UPDATE students
SET major = ?
WHERE name = ?
""", (new_major, student_name))

print("Rows affected:", cur.rowcount)

con.commit()

print("Student major updated successfully.\n")

cur.execute("SELECT * FROM students WHERE name = ?", (student_name,))
print(cur.fetchone())

print("\n===== UPDATE ENROLLMENT GRADE =====")

student_id = 2
course_code = "CS101"
new_grade = "B+"

cur.execute("""
UPDATE enrollments
SET grade = ?
WHERE student_id = ? AND course_code = ?
""", (new_grade, student_id, course_code))

print("Rows affected:", cur.rowcount)

con.commit()

print("Enrollment grade updated successfully.\n")

cur.execute("""
SELECT *
FROM enrollments
WHERE student_id = ? AND course_code = ?
""", (student_id, course_code))

print(cur.fetchone())
```

con.close()

Output:

```
In [11]: runfile('D:/TyBsc39/TY/Assign2 Q4.py', wdir='D:/TyBsc39/TY')
===== UPDATE STUDENT MAJOR =====
Rows affected: 1
Student major updated successfully.

(1, 'Rahul Sharma', 'rahul@gmail.com', 'Data Science')

===== UPDATE ENROLLMENT GRADE =====
Rows affected: 1
Enrollment grade updated successfully.

(3, 2, 'CS101', 'B+')
```

5. Write a Python program to perform delete operations in the SQLite database while handling foreign key constraints.

```
import sqlite3

con = sqlite3.connect("university_assignment.db")
cur = con.cursor()

cur.execute("PRAGMA foreign_keys = ON;")

print("===== DELETE AN ENROLLMENT =====")

student_id = 5
course_code = "DB301"

cur.execute("""
DELETE FROM enrollments
WHERE student_id = ? AND course_code = ?
""", (student_id, course_code))

print("Rows affected:", cur.rowcount)

con.commit()

print("Enrollment deleted successfully.\n")

print("===== ATTEMPT TO DELETE ENROLLED STUDENT =====")

try:
    cur.execute("""
DELETE FROM students
WHERE student_id = ?
""", (1,))

    con.commit()

except sqlite3.IntegrityError as e:
    print("Error:", e)

print()

print("===== SAFE STUDENT DELETION =====")

student_id = 4

cur.execute("""
DELETE FROM enrollments
WHERE student_id = ?
""", (student_id,))

print("Enrollment rows deleted:", cur.rowcount)

cur.execute("""
DELETE FROM students
```

```
WHERE student_id = ?  
""", (student_id,))
```

```
print("Student rows deleted:", cur.rowcount)
```

```
con.commit()
```

```
print("Student and related enrollments deleted successfully.")
```

```
con.close()
```

Output:

```
In [12]: runfile('D:/TyBsc39/TY/Assign2 Q5.py', wdir='D:/TyBsc39/TY')  
===== DELETE AN ENROLLMENT =====  
Rows affected: 1  
Enrollment deleted successfully.  
  
===== ATTEMPT TO DELETE ENROLLED STUDENT =====  
Error: FOREIGN KEY constraint failed  
  
===== SAFE STUDENT DELETION =====  
Enrollment rows deleted: 1  
Student rows deleted: 1  
Student and related enrollments deleted successfully.
```

6. Write a Python program to create full, schema-only, and data-only dumps of the SQLite database into SQL files.

```
import sqlite3
con = sqlite3.connect("university_assignment.db")
cur = con.cursor()
with open("assignment_full_dump.sql", "w") as f:
    for line in con.iterdump():
        f.write("%s\n" % line)

print("Full database dump created: assignment_full_dump.sql")

with open("assignment_schema_only.sql", "w") as f:

    cur.execute("""
    SELECT sql
    FROM sqlite_master
    WHERE type='table'
    AND name IN ('students','enrollments')
    """)

    tables = cur.fetchall()

    for table in tables:
        f.write(table[0] + ";\n\n")

print("Schema-only dump created: assignment_schema_only.sql")

with open("courses_data_dump.sql", "w") as f:

    cur.execute("SELECT * FROM courses")
    records = cur.fetchall()

    for record in records:
        sql = "INSERT INTO courses (course_code, title, credits) VALUES ('{}', '{}', {});\n".format(
            record[0], record[1], record[2]
        )
        f.write(sql)

print("Data-only dump created: courses_data_dump.sql")

con.close()
```

Output:

```
In [13]: runfile('D:/TyBsc39/TY/Assign2 Q6.py', wdir='D:/TyBsc39/TY')
Full database dump created: assignment_full_dump.sql
Schema-only dump created: assignment_schema_only.sql
Data-only dump created: courses_data_dump.sql
```

assignment_full_dump.sql

```
BEGIN TRANSACTION;
CREATE TABLE courses (
    course_code TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    credits INTEGER
);
INSERT INTO "courses" VALUES('CS101','Python Programming',4);
INSERT INTO "courses" VALUES('MA201','Engineering Mathematics',3);
INSERT INTO "courses" VALUES('DB301','Database Management System',4);
CREATE TABLE enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_code TEXT NOT NULL,
    grade TEXT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_code) REFERENCES courses(course_code)
);
INSERT INTO "enrollments" VALUES(1,1,'CS101','A');
INSERT INTO "enrollments" VALUES(2,1,'MA201','B+');
INSERT INTO "enrollments" VALUES(3,2,'CS101','B+');
INSERT INTO "enrollments" VALUES(4,2,'DB301','A');
INSERT INTO "enrollments" VALUES(5,3,'CS101','B');
INSERT INTO "enrollments" VALUES(6,3,'DB301','Pending');
DELETE FROM "sqlite_sequence";
INSERT INTO "sqlite_sequence" VALUES('students',5);
INSERT INTO "sqlite_sequence" VALUES('enrollments',8);
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    major TEXT
);
INSERT INTO "students" VALUES(1,'Rahul Sharma','rahul@gmail.com','Data Science');
INSERT INTO "students" VALUES(2,'Priya Patel','priya@gmail.com','Information Technology');
INSERT INTO "students" VALUES(3,'Amit Verma','amit@gmail.com','Computer Science');
INSERT INTO "students" VALUES(5,'Rohan Mehta','rohan@gmail.com','Information Technology');
COMMIT;
```

assignment_schema_only.sql

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    major TEXT
);

CREATE TABLE enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
student_id INTEGER NOT NULL,  
course_code TEXT NOT NULL,  
grade TEXT,  
FOREIGN KEY (student_id) REFERENCES students(student_id),  
FOREIGN KEY (course_code) REFERENCES courses(course_code)  
);
```

courses_data_dump.sql

```
INSERT INTO courses (course_code, title, credits) VALUES ('CS101', 'Python Programming', 4);  
INSERT INTO courses (course_code, title, credits) VALUES ('MA201', 'Engineering Mathematics', 3);  
INSERT INTO courses (course_code, title, credits) VALUES ('DB301', 'Database Management System', 4);
```

7. Write a Python program to export enrollment data to a CSV file and import new student records from a CSV file into the SQLite database.

```
import sqlite3
import csv

con = sqlite3.connect("university_assignment.db")
cur = con.cursor()

cur.execute("""
SELECT
    enrollments.enrollment_id,
    students.name,
    courses.title,
    enrollments.grade
FROM enrollments
JOIN students
ON enrollments.student_id = students.student_id
JOIN courses
ON enrollments.course_code = courses.course_code
""")

records = cur.fetchall()

with open("enrollments_detailed_export.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["Enrollment ID", "Student Name", "Course Title", "Grade"])

    writer.writerows(records)

print("Enrollment data exported successfully to enrollments_detailed_export.csv")

with open("new_students_to_import.csv", "r") as file:
    reader = csv.reader(file)

    # Skip Header Row
    next(reader)

    for row in reader:
        cur.execute("""
        INSERT INTO students(name, email, major)
        VALUES (?, ?, ?)
        """, row)

con.commit()

print("New students imported successfully.")

print("\nUpdated Students Table:\n")

cur.execute("SELECT * FROM students")
```

```
records = cur.fetchall()
```

```
for record in records:  
    print(record)
```

```
con.close()
```

Output:

```
In [14]: runfile('D:/TyBsc39/TY/Assign2 Q7.py', wdir='D:/TyBsc39/TY')  
Enrollment data exported successfully to  
enrollments_detailed_export.csv  
New students imported successfully.  
  
Updated Students Table:  
  
(1, 'Rahul Sharma', 'rahul@gmail.com', 'Data Science')  
(2, 'Priya Patel', 'priya@gmail.com', 'Information Technology')  
(3, 'Amit Verma', 'amit@gmail.com', 'Computer Science')  
(5, 'Rohan Mehta', 'rohan@gmail.com', 'Information Technology')  
(6, 'Dr. Who', 'timetravel@gmail.com', 'Applied Physics')  
(7, 'Rose Tyler', 'rose@gmail.com', 'Arts')
```

enrollments_detailed_export.csv

Enrollment ID, Student Name, Course Title, Grade

1,Rahul Sharma, Python Programming, A
2,Rahul Sharma, Engineering Mathematics, B+
3,Priya Patel, Python Programming, B+
4,Priya Patel, Database Management System, A
5,Amit Verma, Python Programming, B
6,Amit Verma, Database Management System, Pending

new_students_to_import.csv

name, email, major

Dr. Who, timetravel@gmail.com, Applied Physics

Rose Tyler, rose@gmail.com, Arts